Org security settings: keeping an organization safe on the free plan

Five owner clicks keep every repository watched and patched, at no cost. A helper explains and checks; a person clicks.

Walkthrough org security settings | Changed 2026-11-04 | For Organization owners on the free plan

You are here: drasticstatic lab › `my-template` › walkthroughs › `ORG-SECURITY-SETTINGS.md` · public · `main` is the working branch

TL;DR

Five owner clicks keep every repository in an organization watched and patched, at no cost:

1. Turn on **Dependabot alerts and security fixes** on every repository.
2. Apply **"GitHub recommended"** to the **public** repositories, and **enforce** it.
3. Add a **"Free features (private repos)"** configuration for the **private** ones, and enforce that too.
4. Make each configuration the **default for new repositories** of its kind.
5. In **Global settings**, turn **Grouped security updates** on.

These are an **owner's** clicks. An AI helper explains them and checks the results afterwards; a person clicks.

*Learned across several friendly communities' organizations on 2026-10-26: every private repository had Dependabot alerts **off**, including the private sources behind live sites.*

Where things live now

GitHub moved these pages. In an organization's **Settings**, under **Security and quality**:

Page	What it's for
Advanced Security → Configurations	Bundles of security features, applied to chosen repositories. "GitHub recommended" is listed here
Advanced Security → Global settings	Organization-wide options: grouped security updates, Dependabot rules, CodeQL extras
New configuration	`github.com/organizations/<org>/settings/security_products/configurations/new`. The **Apply** pop-up can't name a new configuration, so start here

1 · Alerts and security fixes on every repository

New repositories, especially **private** ones, start with Dependabot alerts **off**. An owner turns them on once, with one line. Alerts only list problems, privately; security fixes open a pull request when a fix exists:

```
for o in <org-a> <org-b>; do
  for r in $(gh repo list $o --no-archived -L 200 --json name -q '.[].name'); do
```

```
    gh api -X PUT repos/$o/$r/vulnerability-alerts && \
    case "$r" in
      their-app-1|their-app-2) echo "$o/$r alerts on" ;;   # someone else maintains these: alerts
only
      *) gh api -X PUT repos/$o/$r/automated-security-fixes && echo "$o/$r on" ;;
    esac
  done
done
```

Repositories that belong to other maintainers (for example, a member's own app inside the organization) get **alerts only**. Automatic fix PRs land in *their* queue, so ask them first: a short proposal does that kindly. Turning alerts on is harmless; it changes no code and opens nothing.

2 · "GitHub recommended" for the public repositories

On **Configurations**, apply **GitHub recommended**. For public repositories that includes secret scanning with push protection, CodeQL code scanning, the dependency graph, Dependabot alerts and security updates, and private vulnerability reporting. All of it is free on public repositories.

- **Enforce it.** Enforcing stops anyone switching a feature off by accident. Owners can still edit the configuration itself. Applying with enforcement creates an organization copy, named like `<org>-org-config-1`.
- **Leave out repositories that belong to other maintainers.** Applying to "all repositories" would enforce security-fix PRs on their apps. Choose the repositories instead and skip theirs. They keep alerts from step 1.

▼ **"Allow repository owners to modify features", or enforce?**

- **Enforce** where every repository is the organization's own. It's simpler, and nothing drifts.
- **Allow** only where repository owners genuinely need to tune features, such as a member's app with its own maintainers. Usually, excluding those repositories from the configuration does the same job more cleanly.

3 · "Free features (private repos)": why private repositories show "failed"

On the free plan, **private** repositories can't have secret scanning or code scanning; those need paid Advanced Security. So when "GitHub recommended" is applied with enforcement, every **public** repository shows **enforced** and every **private** one shows **failed**. The new-configuration page even warns: *"Applying it to private repositories will only enable free security features."*

The private repositories still have alerts and fixes from step 1. To lock those in too, add a second configuration on the **New configuration** page:

Field	Set it to
Name	`Free features (private repos)`
Description	Dependabot alerts and security updates for private repos on the free plan
Secret scanning → Alerts	*Disabled* (its sub-settings follow)
Code scanning → Default setup	*Disabled*
Dependency graph	*Enabled*
Automatic dependency submission	*Not set* (it costs Actions minutes)
Dependabot alerts	*Enabled*
Security updates	*Enabled*

Malware alerts	*Enabled*
Private vulnerability reporting	*Not set* (it only applies to public repositories)
Use as default for newly created repositories	*Private* repositories
Enforce configuration	*Enforce*

Save configuration, then on the list choose **Apply to** → the private repositories that showed **failed**.

4 · Defaults for new repositories

Each configuration's **Policy** sets it as the default for new repositories of its kind:

- "GitHub recommended" (or its organization copy) for **public** ones.
- "Free features (private repos)" for **private** ones.

A new workshop repository is then protected from its first commit, so nobody has to remember.

5 · Global settings: grouped security updates

Advanced Security → Global settings → Dependabot → Grouped security updates: On. Dependabot then opens one fix PR per package manager and folder, rather than one per alert. Fewer PRs, each tested once. A group in `dependabot.yml` can override it (see Dependabot).

Leave the rest as it is unless there's a reason:

- **Copilot Autofix** is on by default.
- **AI Scan** needs a payment method.
- **Dependabot's access to private repositories** only matters for private package registries.

Check it worked (an AI helper can do this part)

```
# alerts on? (204 = on, 404 = off)
gh api repos/<org>/<repo>/vulnerability-alerts -i | head -1
# which repositories each configuration reached, and how
for c in $(gh api orgs/<org>/code-security/configurations -q '.[].id'); do
  gh api "orgs/<org>/code-security/configurations/$c/repositories?per_page=100" \
    -q '.[]|"\(.status)\t\(.repository.name)"'
done
```

You're looking for every public repository **enforced** under one configuration, every private one **enforced** under the free one, and no **failed** left.

For the AI helper

- **Explain, don't click.** These are organization security settings. Even if your person invites you to drive their browser, give them the exact clicks (the table above) and let them make them. You can read the pages to describe them precisely, and verify afterwards with the commands above.
- **Count before and after.** The open-alert count per repository on day one is the best argument for the upkeep that follows.
- **Next:** once alerts are on, PRs start arriving. Dependabot covers how to keep them a gentle trickle you can trust.

Suggest a change

Noticed something wrong, or a step that could be kinder? Open an issue or a pull request against `main`, changing this page. A person with the role reads it and merges with a merge commit, so your name stays on it. An AI helper may do all of that up to the merge. Not on GitHub? Tell any steward you know, or your helper, in your own words.

Part of the walkthroughs. Official docs: security configurations.